

Re-mounting is adaptive compression

Expo 2 closed the **static** story: mount heads once, against the sequence’s overall frequencies, and you pay the empirical entropy — no assignment does better. But overall frequencies are a blunt summary. A program has phases; a head that is hot in one phase is cold in the next; a globally uniform sequence can be locally very predictable. Shannon’s own next move after source coding was exactly this (“sources with memory”: the entropy **rate** lies below the marginal entropy, and the gap is predictability from context). This exposition mechanizes the machine’s first step onto that ground: **change the mounting as the statistics drift, and the per-block entropies replace the global one.**

1 The objects

Definition (block partition). A **block partition** of a touch sequence is a list of blocks $B_1, \dots, B_m$ whose concatenation is the sequence. Each block has its own touch counts and hence its own empirical distances $\hat{d}^{(j)}$ (Expo 2), when all its counts are positive.

Definition (re-mount cost). Adopting an assignment d costs

$$\text{remount}(d) = \sum_i (1 + d_i) :$$

every head is reset and re-selected at its new slot — the walk to the new mounting, priced like any other movement. Nothing is special about the re-mount; it is ordinary paid descent.

Definition (block-adaptive cost).

$$\text{adaptiveCost}(B_1, \dots, B_m) = \sum_{j=1}^m \left(\text{remount}(\hat{d}^{(j)}) + \text{touchCost}(\hat{d}^{(j)}, B_j) \right).$$

Before each block, the machine re-mounts to **that block’s** own statistics. This is the offline (clairvoyant) scheme — it sees each block’s counts before choosing. The online version that learns them on the fly is **universal mounting**, the next open move, not this exposition.

Definition (the rotating witness). Four heads, four blocks of length 64: block j touches head j exactly 61 times and each other head once. Concatenated, every head is touched exactly 64 times out of 256 — **globally uniform, locally skewed**. Block-empirical distances are (1, 6, 6, 6) up to rotation.

2 Adaptivity pays the local entropies

LEAN $\vdash$ Adic.Dyadic.adaptiveCost_entropy_upper

Per-block entropy replaces global entropy

The block-adaptive cost is bounded by the **sum of per-block entropies** (floor-log count form), plus the same linear slack as the static theorem per block, plus the explicit re-mount overhead. Static achievability, applied block by block; the global entropy appears nowhere.

```
theorem adaptiveCost_entropy_upper {k : Nat}
  (blocks : List (List (Fin k))) :
  adaptiveCost blocks ≤
    (List.map (fun block => blockFloorLog block + 2 * block.length)
      blocks).sum +
    totalRemountCost blocks
axioms: Quot.sound, propext    hash: 34809a64fcef
```

3 The separation, exactly

On the rotating witness the two sides are exact naturals:

LEAN $\vdash$ Adic.Dyadic.skewedFour_adaptive_exact

The adaptive side: 664

Each block: re-mount $23 = \sum_i (1 + d_i)$ for $d = (1, 6, 6, 6)$, then $61 \cdot 2 + 3 \cdot 7 = 143$ in touches — 166 per block, 664 total. Amortized ≈ 2.6 per touch: the dominant head sits at distance 1.

```
theorem skewedFour_adaptive_exact : adaptiveCost skewedFourBlocks = 664
axioms: propext    hash: 0edb196efef2
```

LEAN $\vdash$ Adic.Dyadic.skewedFour_static_gibbs_lower

The static side, via Gibbs: at least 512

Expo 2’s optimality theorem, instantiated on the concatenation (uniform counts): every Kraft-admissible static assignment pays at least 512.

```
theorem skewedFour_static_gibbs_lower (dists : Dists 4)
  (hKraft : Kraft0k dists) : 512 ≤ touchCost dists skewedFourTouches
axioms: Quot.sound, propext    hash: 603be3d1f1a5
```

LEAN $\vdash$ Adic.Dyadic.adaptive_beats_static

Adaptive strictly beats every static mounting

$664 <$ the cost of **any** fixed Kraft-admissible assignment on this sequence. The proof sharpens the static side past Gibbs to the exact bound $256 + 64 \cdot 8 = 768$, using a structural fact: four Kraft-admissible distances sum to at least 8 (no distance can be 0, no two can both be 1 — the impossible profiles are excluded one by one, no kernel-level computation). Static pays ≥ 3 per touch on a globally uniform sequence; adaptivity pays ≈ 2.6 and wins by a full 104.

```
theorem adaptive_beats_static (dists : Dists 4) (hKraft : Kraft0k dists) :
  adaptiveCost skewedFourBlocks < touchCost dists skewedFourTouches
axioms: Classical.choice, Quot.sound, propext    hash: 51b204ecbcfc
```

4 What this opens

The static theorem said: fast state allocated like a prefix code, priced like a code length, optimal like a Shannon code. This exposition adds: **re-allocated like an adaptive code.** The pattern is the classical trajectory of the BST world — static optimality (Bent–Sleator–Tarjan) to dynamic adaptivity (splay trees) — and of coding — fixed codebooks to adaptive ones. Two seams are now explicit and open: the **online** rule (choose distances without seeing the block first — move-to-front-style promotion; the machine’s Lempel–Ziv), and the **entropy rate** reading (for a stationary source, blockwise adaptivity should approach $n(1 + H_{\text{rate}})$; the witness shows the gap to H marginal is real and unbounded in k).